16 — Evaluation & Debugging

Time: ~4-5 hours | **Level**: Advanced

What you'll learn:

- Classification evaluation deep dive: ROC-AUC, PR curves, calibration
- Generation metrics: BLEU, ROUGE, METEOR, BERTScore (hands-on)
- LLM evaluation patterns: LLM-as-judge, rubric scoring, pairwise comparison
- Debugging models: learning curves, gradient analysis, activation distributions
- Common failure modes: data leakage, label noise, distribution shift, shortcut learning
- Overfitting in practice: detection and prevention strategies
- Building automated evaluation harnesses

Prerequisites: Notebooks 02, 07-10 (Classical ML metrics, Transformers, fine-tuning)

Why This Notebook Matters

The difference between a demo and a production model is **evaluation rigour**. A model that looks great in a notebook can fail catastrophically in the real world. This notebook teaches you to catch those failures before your users do.

In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import (
    roc_curve, auc, precision_recall_curve, average_precision_score,
    confusion_matrix, classification_report, f1_score
)
from sklearn.calibration import calibration_curve
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

sns.set_theme(style='whitegrid', font_scale=1.1)
np.random.seed(42)
```

1. Classification Evaluation Deep Dive

Accuracy is a lie on imbalanced data. If 95% of patients are healthy and 5% are at-risk, a model that always predicts "healthy" gets 95% accuracy — but misses every at-risk patient.

The Evaluation Hierarchy

Metric	What It Measures	Use When
Accuracy	Overall correct predictions	Balanced classes only
Precision	Of predicted positives, how many are correct?	Cost of false positives is high
Recall	Of actual positives, how many did we find?	Cost of false negatives is high (healthcare!)
F1	Harmonic mean of precision & recall	Balance between precision and recall
ROC-AUC	Discrimination ability across all thresholds	Comparing models overall
PR-AUC	Performance on the positive class	Imbalanced data (preferred)
Calibration	Does 80% confidence = 80% correct?	When confidence matters for decisions

In [ ]:
```
# ── Generate synthetic mental health classification data ──────────

n_samples = 2000
# Imbalanced: 15% high-risk, 85% low-risk (realistic clinical scenario)
n_positive = int(n_samples * 0.15)
n_negative = n_samples - n_positive

# Features: depression_score, anxiety_score, sleep_quality, social_support
X_pos = np.random.normal(loc=[0.7, 0.65, 0.3, 0.25], scale=0.15, size=(n_pos
X_neg = np.random.normal(loc=[0.3, 0.25, 0.7, 0.7], scale=0.2, size=(n_negat

X = np.vstack([X_pos, X_neg])
y = np.array([1] * n_positive + [0] * n_negative)

# Shuffle and split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, rar

# Train two models for comparison
lr = LogisticRegression(random_state=42).fit(X_train, y_train)
rf = RandomForestClassifier(n_estimators=100, random_state=42).fit(X_train,

# Get predicted probabilities
y_prob_lr = lr.predict_proba(X_test)[:, 1]
y_prob_rf = rf.predict_proba(X_test)[:, 1]

print(f'Dataset: {n_samples} patients ({n_positive} high-risk, {n_negative}
print(f'Class balance: {n_positive/n_samples:.1%} positive (imbalanced)')
print(f'Train: {len(X_train)}, Test: {len(X_test)}')
print(f'\nLogistic Regression accuracy: {lr.score(X_test, y_test):.3f}')
print(f'Random Forest accuracy: {rf.score(X_test, y_test):.3f}')
print(f'\n💡 Both look good by accuracy. But let\'s dig deeper...')
```

In [ ]:
```
# ── ROC curves, PR curves, and calibration ──────────────────────
```

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# ROC Curve
for name, y_prob, color in [('Logistic Regression', y_prob_lr, '#3498db'),
                            ('Random Forest', y_prob_rf, '#e74c3c')]:
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    roc_auc = auc(fpr, tpr)
    axes[0].plot(fpr, tpr, color=color, lw=2, label=f'{name} (AUC={roc_auc:.
axes[0].plot([0, 1], [0, 1], 'k--', lw=1, label='Random (AUC=0.500)')
axes[0].set_xlabel('False Positive Rate')
axes[0].set_ylabel('True Positive Rate')
axes[0].set_title('ROC Curve')
axes[0].legend(fontsize=9)

# Precision-Recall Curve
for name, y_prob, color in [('Logistic Regression', y_prob_lr, '#3498db'),
                            ('Random Forest', y_prob_rf, '#e74c3c')]:
    prec, rec, _ = precision_recall_curve(y_test, y_prob)
    ap = average_precision_score(y_test, y_prob)
    axes[1].plot(rec, prec, color=color, lw=2, label=f'{name} (AP={ap:.3f})'
axes[1].axhline(y=y_test.mean(), color='k', linestyle='--', lw=1, label=f'Ba
axes[1].set_xlabel('Recall')
axes[1].set_ylabel('Precision')
axes[1].set_title('Precision-Recall Curve (better for imbalanced data)')
axes[1].legend(fontsize=9)

# Calibration plot
for name, y_prob, color in [('Logistic Regression', y_prob_lr, '#3498db'),
                            ('Random Forest', y_prob_rf, '#e74c3c')]:
    prob_true, prob_pred = calibration_curve(y_test, y_prob, n_bins=10)
    axes[2].plot(prob_pred, prob_true, 's-', color=color, label=name)
axes[2].plot([0, 1], [0, 1], 'k--', lw=1, label='Perfectly calibrated')
axes[2].set_xlabel('Mean Predicted Probability')
axes[2].set_ylabel('Fraction of Positives')
axes[2].set_title('Calibration Plot')
axes[2].legend(fontsize=9)

plt.tight_layout()
plt.show()

print('💡 Key observations:')
print('  • PR curve is more informative than ROC for imbalanced data')
print('  • Logistic Regression is naturally well-calibrated (sigmoid output)
print('  • Random Forest tends to be poorly calibrated (needs CalibratedClas
```

2. Generation Evaluation Metrics

For text generation (clinical summaries, assessments), we need different metrics:

BLEU Score

Measures **n-gram precision** — what fraction of generated n-grams appear in the reference?

$$\text{BLEU} = BP \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right)$$

ROUGE Scores

- **ROUGE-1**: Unigram overlap (individual words)
- **ROUGE-2**: Bigram overlap (word pairs)
- **ROUGE-L**: Longest common subsequence (captures sentence structure)

BERTScore

Uses contextual embeddings to compute **semantic similarity** between generated and reference text. Better than n-gram metrics for paraphrases.

In [ ]:
```
# ── Compute ROUGE scores on clinical text ──────────────────────
from rouge_score import rouge_scorer

scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'], use_stemme

# Reference clinical assessments vs model-generated ones
eval_pairs = [
    {
        'reference': 'Patient presents with moderately severe depression. PH
        'generated': 'The patient shows signs of moderate to severe depressi
        'label': 'Good generation (paraphrased)'
    },
    {
        'reference': 'Patient presents with moderately severe depression. PH
        'generated': 'Patient seems fine. No immediate concerns noted. Conti
        'label': 'Bad generation (missed severity)'
    },
    {
        'reference': 'Suicidal ideation present. Immediate risk assessment r
        'generated': 'Active suicidal thoughts detected. Urgent: conduct C-S
        'label': 'Good generation (clinical synonym)'
    }
]

print('=== ROUGE Scores for Clinical Text Generation ===\n')
for pair in eval_pairs:
    scores = scorer.score(pair['reference'], pair['generated'])
    print(f'{pair["label"]}:')
    print(f'  ROUGE-1: P={scores["rouge1"].precision:.3f} R={scores["rouge1"
    print(f'  ROUGE-2: P={scores["rouge2"].precision:.3f} R={scores["rouge2"
    print(f'  ROUGE-L: P={scores["rougeL"].precision:.3f} R={scores["rougeL"
    print()

print('💡 ROUGE catches the "bad" generation (low scores) but struggles with
print('   "SSRI medication" vs "antidepressant medication" — same meaning, o
print('   That\'s where BERTScore helps (semantic similarity vs n-gram overl
```

3. LLM Evaluation Patterns

Automated metrics (ROUGE, BERTScore) **correlate poorly with human judgement** for open-ended generation.

LLM-as-judge uses a strong model to evaluate outputs on specific criteria:

Method Comparison

Method	Scalability	Cost	Correlation with Humans
Human evaluation	Low (expensive, slow)	High	1.0 (by definition)
LLM-as-judge	High	Medium	0.7-0.9
ROUGE/BLEU	Very high	Very low	0.3-0.5
BERTScore	High	Low	0.5-0.7

Known Biases in LLM-as-Judge

- **Position bias**: In pairwise comparison, prefers the response shown first
- **Verbosity bias**: Rates longer responses higher, even when less accurate
- **Self-preference**: GPT-4 rates its own outputs higher than Claude's (and vice versa)

In [ ]:
```python
# ── Build an evaluation harness ──────────────────────────────────

class EvaluationHarness:
    """Automated evaluation pipeline for clinical NLP models."""

    def __init__(self):
        self.rouge_scorer = rouge_scorer.RougeScorer(
            ['rouge1', 'rouge2', 'rougeL'], use_stemmer=True
        )
        self.results = []

    def evaluate_generation(self, references, predictions):
        """Compute all generation metrics."""
        rouge_scores = {'rouge1': [], 'rouge2': [], 'rougeL': []}

        for ref, pred in zip(references, predictions):
            scores = self.rouge_scorer.score(ref, pred)
            for key in rouge_scores:
                rouge_scores[key].append(scores[key].fmeasure)

        return {
            'rouge1_f': np.mean(rouge_scores['rouge1']),
            'rouge2_f': np.mean(rouge_scores['rouge2']),
            'rougeL_f': np.mean(rouge_scores['rougeL']),
            'n_samples': len(references)
        }
```

```
    def evaluate_classification(self, y_true, y_pred, y_prob=None):
        """Compute all classification metrics."""
        results = {
            'f1_macro': f1_score(y_true, y_pred, average='macro'),
            'f1_weighted': f1_score(y_true, y_pred, average='weighted'),
        }
        if y_prob is not None:
            fpr, tpr, _ = roc_curve(y_true, y_prob)
            results['roc_auc'] = auc(fpr, tpr)
            results['avg_precision'] = average_precision_score(y_true, y_pro
        return results

    def run_full_evaluation(self, y_true, y_pred, y_prob, references, predic
        """Run all evaluations and return a comprehensive report."""
        cls_metrics = self.evaluate_classification(y_true, y_pred, y_prob)
        gen_metrics = self.evaluate_generation(references, predictions)
        return {**cls_metrics, **gen_metrics}

# Demo the harness
harness = EvaluationHarness()

# Classification evaluation
cls_results = harness.evaluate_classification(
    y_test, rf.predict(X_test), rf.predict_proba(X_test)[:, 1]
)

# Generation evaluation
refs = [p['reference'] for p in eval_pairs]
preds = [p['generated'] for p in eval_pairs]
gen_results = harness.evaluate_generation(refs, preds)

print('=== Evaluation Harness Results ===')
print('\nClassification Metrics:')
for k, v in cls_results.items():
    print(f'  {k}: {v:.4f}')
print('\nGeneration Metrics:')
for k, v in gen_results.items():
    print(f'  {k}: {v:.4f}' if isinstance(v, float) else f'  {k}: {v}')

print('\n💡 Build your evaluation harness BEFORE training. Run it after ever
```

4. Debugging Models — Detective Work

Your model isn't working. Here's the debugging flowchart:

```
Training loss not decreasing?
  → LR too high (diverging) or too low (stuck)
  → Bug in data pipeline (wrong labels, wrong features)
  → Model too small for the task

Training loss low, validation loss high?
  → OVERFITTING. Add regularisation, get more data, simplify
```

```
    model.

    Both losses low, but test performance bad?
      → DATA LEAKAGE or distribution shift.

    Model works on test set, fails in production?
      → Distribution shift between test data and real-world data.
```

In [ ]:

```python
# ── Learning curves: the first debugging tool ──────────────────
import torch
import torch.nn as nn

# Simple neural network for demonstration
class SimpleNet(nn.Module):
    def __init__(self, input_dim, hidden_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, 1),
            nn.Sigmoid()
        )
    def forward(self, x):
        return self.net(x).squeeze()

def train_and_track(model, X_train, y_train, X_val, y_val, epochs=100, lr=0.
    """Train model and track learning curves."""
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    criterion = nn.BCELoss()

    X_tr = torch.FloatTensor(X_train)
    y_tr = torch.FloatTensor(y_train)
    X_v = torch.FloatTensor(X_val)
    y_v = torch.FloatTensor(y_val)

    train_losses, val_losses = [], []

    for epoch in range(epochs):
        model.train()
        optimizer.zero_grad()
        pred = model(X_tr)
        loss = criterion(pred, y_tr)
        loss.backward()
        optimizer.step()
        train_losses.append(loss.item())

        model.eval()
        with torch.no_grad():
            val_pred = model(X_v)
            val_loss = criterion(val_pred, y_v)
            val_losses.append(val_loss.item())

    return train_losses, val_losses
```

```
# Train 3 models with different complexities to show overfitting
fig, axes = plt.subplots(1, 3, figsize=(16, 4))
configs = [
    ('Small (8 hidden)', 8),
    ('Medium (32 hidden)', 32),
    ('Large (256 hidden)', 256),
]

for ax, (name, hidden) in zip(axes, configs):
    torch.manual_seed(42)
    model = SimpleNet(4, hidden)
    train_l, val_l = train_and_track(model, X_train, y_train, X_test, y_test

    ax.plot(train_l, label='Train Loss', color='#3498db')
    ax.plot(val_l, label='Val Loss', color='#e74c3c')
    ax.set_title(name)
    ax.set_xlabel('Epoch')
    ax.set_ylabel('Loss')
    ax.legend(fontsize=9)
    ax.grid(True, alpha=0.3)

plt.suptitle('Learning Curves: Detecting Overfitting', fontsize=14, y=1.02)
plt.tight_layout()
plt.show()

print('💡 Large model: train loss keeps dropping but val loss increases = OV
print('   Small model: both losses plateau high = UNDERFITTING.')
print('   Medium model: both losses converge = GOOD FIT.')
```

5. Common Failure Modes

Data Leakage — The Silent Killer

Data leakage = information from the test set leaks into training, giving unrealistically good results.

Types:

1. **Feature leakage**: A feature that encodes the target (e.g., "prescribed_antidepressant" when predicting depression)
2. **Train-test contamination**: Same patient appears in both train and test sets
3. **Temporal leakage**: Using future data to predict past events
4. **Preprocessing leakage**: Fitting scaler/encoder on the full dataset, not just training

In [ ]:
```
# ── Data leakage demonstration ──────────────────────────────

# Create data WITH leakage: add a feature that directly encodes the target
X_leaked = np.column_stack([
    X,  # original features
    y + np.random.normal(0, 0.1, len(y))  # leaked feature: noisy version of
])
```

```
X_clean = X  # original features without leakage

# Train on leaked vs clean data
X_tr_leak, X_te_leak, y_tr_leak, y_te_leak = train_test_split(
    X_leaked, y, test_size=0.3, random_state=42, stratify=y
)

model_leaked = LogisticRegression(random_state=42).fit(X_tr_leak, y_tr_leak)
model_clean = LogisticRegression(random_state=42).fit(X_train, y_train)

f1_leaked = f1_score(y_te_leak, model_leaked.predict(X_te_leak))
f1_clean = f1_score(y_test, model_clean.predict(X_test))

print('=== Data Leakage Detection ===')
print(f'\nF1 score WITHOUT leakage: {f1_clean:.3f}')
print(f'F1 score WITH leakage:    {f1_leaked:.3f}  ← suspiciously high!')
print(f'\nDifference: +{f1_leaked - f1_clean:.3f}')

print(f'\n🚨 Red flags for data leakage:')
print(f'   • Test performance is "too good to be true" (F1 > 0.95 on real cl
print(f'   • Model performs much worse in production than on test set')
print(f'   • One feature has extremely high importance')

# Show feature importance to detect the leaked feature
feature_names = ['depression', 'anxiety', 'sleep', 'social_support', 'LEAKED
importances = np.abs(model_leaked.coef_[0])

fig, ax = plt.subplots(figsize=(10, 4))
colors = ['#3498db'] * 4 + ['#e74c3c']
ax.barh(feature_names, importances, color=colors)
ax.set_xlabel('Absolute Coefficient')
ax.set_title('Feature Importance — Can You Spot the Leakage?')
plt.tight_layout()
plt.show()

print('\n💡 The leaked feature dominates — a clear red flag.')
print('   Always investigate features with suspiciously high importance.')
```

6. Overfitting in Practice

Overfitting = model memorises training data instead of learning generalisable patterns.

Detection

- Training accuracy 99%, validation accuracy 60% → memorisation
- Validation loss starts increasing while training loss continues decreasing

Prevention

Technique	How It Helps	Used In
Dropout	Randomly drops neurons during training	Neural networks
Weight decay (L2)	Penalises large weights	All models
Early stopping	Stop training when val loss plateaus	All iterative models
Data augmentation	Increases effective dataset size	Images, text
Ensemble methods	Average multiple models	All models
Reduce model size	Fewer parameters to memorise with	Neural networks

In [ ]:

```
# ── Regularisation comparison ──────────────────────────────────

class RegularisedNet(nn.Module):
    def __init__(self, input_dim, hidden_dim, dropout=0.0):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden_dim, 1),
            nn.Sigmoid()
        )
    def forward(self, x):
        return self.net(x).squeeze()

# Compare: No regularisation vs Dropout vs Weight Decay vs Both
configs = [
    ('No regularisation', 0.0, 0.0),
    ('Dropout (0.3)', 0.3, 0.0),
    ('Weight decay (0.01)', 0.0, 0.01),
    ('Dropout + Weight decay', 0.3, 0.01),
]

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

for ax, (name, dropout, wd) in zip(axes.flatten(), configs):
    torch.manual_seed(42)
    model = RegularisedNet(4, 128, dropout=dropout)
    optimizer = torch.optim.Adam(model.parameters(), lr=0.01, weight_decay=w
    criterion = nn.BCELoss()

    X_tr_t = torch.FloatTensor(X_train)
    y_tr_t = torch.FloatTensor(y_train)
    X_te_t = torch.FloatTensor(X_test)
    y_te_t = torch.FloatTensor(y_test)

    train_losses, val_losses = [], []
    for epoch in range(200):
        model.train()
        optimizer.zero_grad()
```

```
        loss = criterion(model(X_tr_t), y_tr_t)
        loss.backward()
        optimizer.step()
        train_losses.append(loss.item())

        model.eval()
        with torch.no_grad():
            val_losses.append(criterion(model(X_te_t), y_te_t).item())

    ax.plot(train_losses, label='Train', color='#3498db')
    ax.plot(val_losses, label='Validation', color='#e74c3c')
    gap = val_losses[-1] - train_losses[-1]
    ax.set_title(f'{name}\n(gap={gap:.3f})')
    ax.set_xlabel('Epoch')
    ax.set_ylabel('Loss')
    ax.legend(fontsize=9)
    ax.grid(True, alpha=0.3)

plt.suptitle('Regularisation Effects on Overfitting', fontsize=14)
plt.tight_layout()
plt.show()

print('💡 Regularisation reduces the gap between training and validation los
print('   Combining dropout + weight decay gives the best generalisation.')
```

7. Building an Evaluation Pipeline

Best practices for evaluation:

1. **Build the harness before training** — know what you're optimising for
2. **Automate everything** — run all metrics in one pass
3. **Track over time** — store results to compare across experiments
4. **Regression testing** — ensure model updates don't degrade existing capabilities
5. **Multiple metrics** — no single metric tells the whole story

Evaluation Checklist

- [ ] Classification: F1, ROC-AUC, PR-AUC, calibration
- [ ] Generation: ROUGE, BERTScore
- [ ] Fairness: performance across demographic groups
- [ ] Robustness: performance on adversarial/edge cases
- [ ] Latency: inference time at various batch sizes
- [ ] Human evaluation: sample-based quality review

In [ ]:
```
# ── Comparing models across experiments ──────────────────────────

# Simulated experiment results over time
experiments = {
    'Baseline (LR)':      {'f1': 0.72, 'roc_auc': 0.85, 'rouge1': 0.0, 'late
    'Random Forest':      {'f1': 0.78, 'roc_auc': 0.89, 'rouge1': 0.0, 'late
```

```
    'Fine-tuned Phi-3':   {'f1': 0.83, 'roc_auc': 0.91, 'rouge1': 0.45, 'lat
    'Phi-3 + RAG':        {'f1': 0.85, 'roc_auc': 0.92, 'rouge1': 0.52, 'lat
    'Phi-3 + RAG + eval': {'f1': 0.87, 'roc_auc': 0.93, 'rouge1': 0.58, 'lat
}

# Visualise improvement trajectory
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

exp_names = list(experiments.keys())
metrics = ['f1', 'roc_auc', 'latency_ms']
titles = ['F1 Score', 'ROC-AUC', 'Latency (ms)']
colors_list = ['#3498db', '#2ecc71', '#e74c3c']

for ax, metric, title, color in zip(axes, metrics, titles, colors_list):
    values = [experiments[name][metric] for name in exp_names]
    ax.bar(range(len(exp_names)), values, color=color, alpha=0.8)
    ax.set_xticks(range(len(exp_names)))
    ax.set_xticklabels(exp_names, rotation=45, ha='right', fontsize=8)
    ax.set_title(title)
    ax.grid(True, alpha=0.3, axis='y')

plt.suptitle('Model Improvement Trajectory', fontsize=14, y=1.02)
plt.tight_layout()
plt.show()

print('💡 Each experiment improved quality (F1, AUC) at the cost of latency.
print('   The evaluation harness makes this tradeoff visible and quantified.
print('   Always track ALL metrics — optimising one can silently degrade anc
```

🧪 Exercises

1. **Data leakage detection**: Create a deliberately leaky train/test split (include the same patients in both sets) and show how F1 jumps unrealistically. Then fix the split and compare.

2. **Early stopping**: Implement early stopping with patience=10 that saves the best model checkpoint based on validation loss. Train for 500 epochs and see when it actually stops.

3. **Evaluation harness**: Extend the `EvaluationHarness` class to also compute BERTScore (using the `bert-score` library) and add a method to save results as JSON.

4. **Learning curves by model size**: Train 5 models with hidden dimensions [4, 16, 64, 256, 1024]. Plot the final train/val gap vs model size. At what point does overfitting become severe?

5. **Fairness evaluation**: Split the test set by a demographic variable (e.g., age groups: <30, 30-50, >50). Report F1 per group. Is the model equally accurate across demographics?

✅ Key Takeaways

1. **Accuracy alone is misleading** — always check ROC-AUC, PR curves, and calibration on imbalanced data
2. **ROUGE and BERTScore** are standard for generation evaluation; BLEU is mostly for translation
3. **LLM-as-judge** scales evaluation but watch for position, verbosity, and self-preference biases
4. **Learning curves** are your first debugging tool — they reveal overfitting, underfitting, and data issues
5. **Data leakage** is the #1 cause of "too good to be true" results — always investigate suspiciously high metrics
6. **Regularisation** (dropout + weight decay) reduces the train-val gap and improves generalisation
7. **Build evaluation harnesses early** — they pay for themselves throughout development

Next: 17 — MLOps & Experiment Tracking — tracking experiments, versioning models, and monitoring in production